

(1)

Aim:

To implement the K-nearest neighbors (KNN) algorithm to classify data based on similarity and to evaluate its performance using a given dataset.

Algorithm:

- ⇒ Separate Features (x) and target labels (y).
- ⇒ Split the dataset into training and testing data.
- ⇒ Choose the value of K (no. of neighbors).
- ⇒ Calculate the distance b/w the test point and all training points.
- ⇒ Select the K nearest neighbors based on minimum distance.
- ⇒ Assign the class label based on majority.
- ⇒ Evaluate the model using accuracy score.

Program:

```
import pandas as pd.  
from sklearn.model_selection import  
train_test_split  
from sklearn.neighbors import  
KNeighborsClassifier  
from sklearn.metrics import accuracy_score.  
data = pd.read_csv ("job_data.csv")  
X = data.drop ('Job', axis=1)  
y = data ['Job']
```

Output: 0.86 (Accuracy)

Result: The K-nearest neighbors algorithm was successfully implemented and used to classify the dataset.

(2) Aim: To implement the naive Bayes algorithm for classification and evaluate its performance on a given dataset.

Algorithm:

- ⇒ Load the dataset
- ⇒ separate features and target labels
- ⇒ split the dataset into training and testing sets.
- ⇒ Apply the naive Bayes classifier.
- ⇒ Train the model using training data.
- ⇒ predict class labels for test data.
- ⇒ calculate accuracy of the model.

Program:
import pandas as pd
from sklearn.model_selection import
train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score.

```
data = pd.read_csv("data.csv")  
x = data.drop('class', axis=1)  
y = data['class']  
x_train, x_test, y_train, y_test = train_test_split(  
    x, y, test_size=0.3, random_state=1)
```

output: 0.888 (Accuracy)

Result: The naive Bayes classifier was successfully implemented and achieved good accuracy.

(3) Aim: To implement a decision tree algorithm for classification and analyze its performance.

Algorithm:

- ⇒ Load the dataset
- ⇒ Divide data into training and testing sets.
- ⇒ Split the dataset into training and testing sets.
- ⇒ Build a decision tree model.
- ⇒ Train the model using training data.
- ⇒ Predict outcomes for test data.
- ⇒ Measure accuracy.

Program: Import pandas as pd.

from sklearn.model_selection import

train_test_split.

from sklearn.tree import DecisionTreeClassifier.

from sklearn.metrics import accuracy_score.

data = pd.read_csv("data.csv")

X = data.drop('class', axis=1)

X = data.drop('class', axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

Output: 0.91 Accuracy:

Result: The decision tree model was successfully implemented and produced high accuracy, showing its capability to handle classification tasks efficiently.

(4) Aim: To implement support vector machine (svm) for classification and evaluate its accuracy.

Algorithm:

- ⇒ Load the dataset.
- ⇒ Separate independent and dependent variables.
- ⇒ Split the dataset into training and testing sets.
- ⇒ Apply svm classifier with suitable kernel.
- ⇒ Train the model.
- ⇒ Predict test data labels.
- ⇒ Calculate accuracy.

Program: Import pandas as pd

```
from sklearn.model_selection import  
train_test_split  
from sklearn.svm import SVC  
from sklearn.metrics import accuracy_score  
data = pd.read_csv("data.csv")  
X = data.drop('class', axis=1)  
Y = data['class']  
X_train, X_test, Y_train, Y_test = train_test_split(  
X, Y, test_size=0.3, random_state=0)
```

Output: 0.89. Accuracy.

Result: The support vector machine classifier was successfully implemented and achieved good accuracy, demonstrating its effectiveness in classification problems.